Alfredo De La Torre
Project_Design_CMPE132_Fall2024_Ver1
10/13/24

Part 1.

- Well-crafted 350-word introduction to your model, whether you used RBAC, DAC, or MAC. Note: do not use Attribute Based Access Control (ABAC). ABAC will be a separate part of the project.
- Describe the criteria and decision processes you went through to create that model.
- Provide a diagram of the model.
- Describe how you are implementing your model (RBAC, DAC, MAC); specifically, how your model is implemented: matrix, access-lists, capability lists. Describe how your model is represented in the database

RBAC (Role Based Access Control) is the implementation of simple and easy management of giving permissions and roles to different users. A pro of using the RBAC model for the library would allow easy management for roles that many users are going to be put under, and there will be a lot of those kinds of users when implementing the SJSUL based access control system. Another pro of using RBAC is the flexibility it has because as large as the system becomes the more roles needed to be added, but RBAC solves that issue. In my Library (SJSU_Library (SJSUL)) RBAC would be the ideal based access control model since our main goal is to implement a web-based system to perform primary user enrollment, authentication, and authorization functions for SJSUL. When deciding which kind of based access control model I should implement I thought about the criteria of having to define the roles for very specific tasks needed for those users, modifications of people's roles are easy to do, and there is a good balance in security and flexibility in the system. RBAC fits all those criterias, which leads me to my final step in making my decision.

User ID #	Users
U_1	Jeff
U_2	John
U_3	Jeb
U_4	kyle
U_5	Julio

Role ID #	Roles
R_1	Librarian
R_2	Administrator
R_3	IT support
R_4	Library member
R_5	researcher

Permission ID #	Permissions
P_1	Add/Remove books
P_2	Access entire library
P_3	Borrow books
P_4	Acces databases for research purposes
P_5	Update book information
P_6	Manage user accounts/roles
P_7	Place holds/fine on accounts
P_8	Manage their own accounts
P_9	Limited from library resources
P_{10}	Shelving books
P_{11}	Access system logs/ settings/ repors
P_{12}	Provides IT support to staff and members
P_{13}	No access to other accounts
P_{14}	View book details
P_{15}	Loan/borrowing permissions

User ID #	Role ID #	Role name	Permissions
U_1	R_1	Librarian	$P_1, P_2, P_3, P_4, P_5, P_7, P_8, P_{10}, P_{13}, P_{14}, P_{15}$
U_2	R_2	Administrator	$P_2, P_3, P_4, P_6, P_8, P_{11}, P_{14}$
U_3	R_3	IT support	$P_4, P_6, P_8, P_{11}, P_{12}, P_{14}$
U_4	R_4	Library member	P_3, P_8, P_{13}, P_{14}
U_5	R_5	researcher	$P_3, P_4, P_8, P_{13,14}$

How are we going to implement everything in the RBAC model? RBAC has three main components, which is users, roles, and permissions. And the implementation would be based on those three roles. Some examples of user roles that could be implemented are Administrator, Librarian, Librarian Assistant, IT specialists, researcher, Librarian member, guest, and volunteer. With those roles we would first put them under a role ID number, and create specific positions with specific permission ID numbers. Then, based on the role ID numbers we would assign what kind of possessions with the permission ID numbers.

In the database the model would be represented through matrices. When the user is assigned a role they would have to make a username and password and the password would be encrypted and stored away. The password would be used for authentication purposes. With each

role they are assigned specific permissions ID numbers and role ID numbers. The Matrix would help connect each user with their proper roles and permissions, and make it easier to verify if the user is allowed to do those specific tasks.

Part 2.

- Define at least three use cases.
- Document how each use case is mapped to your access control model.
- Document the mapping of each use case. The use cases must define subjects, objects, permissions, capabilities, constraints, and inheritances.
- Your work should be free of grammar and usage errors.

Case 1:

Situation:

A library member wants to reserve a book and get a week extension of the book. The system would first have to check if it is available, if it is available then the staff would lend the book and set the book as borrowed.

The mapping:

Role: Librarian Member
Permissions: Borrow Book, Access the system of books to look the information
Inheritance: Researchers inherit Borrow Books and have access to special materials.
Object: Book, User Account information
Capabilities: Search in the system, borrow books.
Constraints: Limited by book availability, borrowing limits, and account standing.

Subjects:

- Library Member - Wants to borrow a book and have an extension

Objects:

- Book
- Account information

Permissions:

- Borrow books
- Allowed to request extensions
- Able to access the system to search for books

Capabilities:

- Has access to see the book's details
- If the book is available in the system then the member can borrow the book

Constraints:

- Depending on what kind of book it is, the library member might not be given access to borrow it. For instance, if it is a book under the categories for research and only users who have researcher permissions can borrow those books.

- Cannot ask for the extension of the book depending on the book
- The user must be in good standing to borrow books and cannot borrow more books than allowed

Inheritances:

- The researcher role, and Librarian role are roles that inherits the borrow book permissions for specific books, and can request extensions on those books

Case 2:

Situation:

- A Library member is needs help in resetting their password, so they go to IT Support in the library for assistance

The mapping:

Role: IT Support
Permissions: Reset login information, configure the library's system and security, remove accounts, and keep track of logs and audit
Inheritance: The librarian and Administrator has access to look at databases and
Object: User Account
Capabilities: Access multiple databases in the library, can reset login information for users, configuring the system and it's system
Constraints: Cannot change any account information that they don't have permission to change, cannot change any information about books in the library system, and cannot add new users to the system.

Subjects:

- IT Support - Helping a member in resetting their password

Objects:

- User account
- Access to user's account to reset password

Permissions:

- Access to different accounts reset passwords and other information for user login purposes

Capabilities:

- Has access almost complete access of the library system
- Can keep track of logs and audits
- Can reset login information for different users
- Has the ability to do system and security configurations

- Can access multiple databases in the system

Constraints:

- Not able to add or remove books in the system
- Cannot rent out books as a IT Support only and needs to be considered to be a library member in the system or have another account that is considered to be a library account

Inheritances:and manage user accounts, but the Administrator can also access the shole library system, look at system and security logs,

- The Administrator and Librarian role has similar positions like the IT Support roles. Such as, being able to access databases and manage user accounts

Case 3:

Situation:

A librarian updating the system by adding or removing books

The mapping:

Role: Librarian
Permissions: Add/remove book with permission from admin
Inheritance: Administrators can do the same a librarian, but can take action themselves when adding new or removing books
Object: Library System
Capabilities: Adding/removing books, and organizing the books in their proper locations
Constraints: Cannot modify current existing books or add and remove books in the system without permission

Subjects:

- Librarian

Objects:

- Access to the Library system to make changes

Permissions:

- Able to add or remove books from the library's system with permission

Capabilities:

- Add/remove books in the system with permission
- Organize where the books are located in the library
- Add details about the book to the system

Constraints:

- Needs permission to remove existing books from the system from administrators
- Only people like the Librarian and the Administrator can add/remove books in the system

Inheritances:

- The administrator has access to the same permissions as a Librarian, but only an administrator can modify existing books and give those specific tasks to the librarian. They cannot do it themselves without permission from the administrator.

Part 3.

Access Control Policy Testing and Validation

- Describe a set of test cases to validate their access control schema against potential security vulnerabilities, such as privilege escalation, unauthorized access, and role conflicts.
- The objective is to think critically about the robustness of their access control model.
- Write a test plan with five detailed test cases and expected results.

Test case 1:

- Privilege escalation

Summary:

Making sure the “lowest privileged” cannot change any of their permissions to gain access to specific information and functions they shouldn’t have access to.

Testing Process:

1. Make a brand new account which is considered to be the “least privileged”
2. Once logged in we will try to see if we have higher level functions. If not, we try messing with the URL and see if manipulating it would give you access to higher level functionality.
3. Go into your settings and see if you can select some kind of options to give administrator access or functions
4. Lastly, check if there is any changes gave the account special functions that normal accounts don’t have

Results:

- Any changes should not give the account administrator access and functions, and it shouldn't be logged in as an administrator at any point.
- There should be functions that allows the user to escalate their privileges

Test case 2:

Verification and authorization

Summary:

Verifying that using the wrong credentials for logging into a library members' account wouldn't let you access it. Also making sure that all roles have authorization to use their specific permissions given to them and can't access other user's permissions.

Testing Process:

1. Log into an account using the wrong credentials to see the login process works and they are not allowed to access the account without the correct credentials
2. After logging into the account the users cannot access permissions not meant for their roles and their specific roles work
3. Lastly make sure all the higher level roles work as intended and their permissions are working properly

Results:

- If a librarian logs into their account with the right credentials they should be able to access their account and use all the functions a librarian has
- Users can only use their specific permissions based on their roles and cannot access other roles' permissions that are not assigned to them.

Test case 3:

- Role conflict

Summary:

. Users assigned with multiple roles are not having conflicts with their permissions and having access to information they shouldn't have.

Testing Process:

1. Have a user be assigned two different roles. For this instance we will use librarian and IT Support
2. Start testing all the permissions of a librarian logged in as IT Support, and vice-versa.
3. Then see if both roles' permissions work at the same time
4. Then attempt to use roles that librarians and IT Supports don't have access to. Such as, trying to use administration permissions.

Results:

- The user with multiple roles should only be able to access the roles permissions, run smoothly, and don't have access to other roles' permissions.

Test case 4:

- Permission boundaries

Summary:

Ensuring that users cannot access other users' roles. Like, a researcher cannot use the same permissions as a librarian or a IT Support

.

Testing Process:

1. Go into the system and and log in as a researcher
2. Ensure that a researcher have access to books that only researchers can access
3. Attempt to use librarian permissions and see if they work
4. Then try to access IT Support permissions while still have the researcher role

Results:

- Users should not be able to access other users' permissions without having the role.
- Everything should be restricted if any attempts of permissions are made and not having those specific permissions in your roles.

Test case 5:

- Permission revocation

Summary:

. Users whose roles are revoked should not have access to their permissions after being removed. Therefore, ensuring the system management works as intended.

Testing Process:

1. Set a user as a librarian role and allow that user to use the permissions of a librarian.
2. Now revoke the librarian role from the user and see if the user can still use librarian functions
3. Have the user logout and log back in. Then the user will attempt to use librarian permissions

Results:

- After a user's role has been revoked they should instantaneously lose all access to that role and their permissions. Regardless if they are logged in or logged out.

Part 4.

Attribute Based Access Control (ABAC) or dynamic role adjustment based on context

- With a well-crafted 250-word description of how your system can be extended to change authorization based on context, such as:
    - location, time of day, or specific user behavior.
- Describe the mechanism you will use to represent the additional attributes: tags, ACLs, capability lists, matrix or other method.
- The objective is to consider more advanced and real-world scenarios where roles and permissions aren't static.

- Describe how you are implementing ABAC, specifically, how ABAC will be implemented:
    - tags, matrix, access-lists, capability lists. Describe how ABAC will be represented in the database.

The SJSUL (San Jose State University Library system) can be extended to support context-based authorization through the implementation of ABAC (Attribute Based Access Control). ABAC helps change authorization based on specifics like, the location, the time of day, and the user's behavior. It could lead to it being more complex, but it leads to greater security. The mechanisms that I will implement are matrices and and tags. The reason for tags and matrices is because tags can help with knowing what kind of access does users get to have. For instance, for location we would have tags like remote, at-site or home, for time of day tags would be on-the-clock or off-the-clock, and for behavior tags it would be low-risk or high risk. With a matrix it would be implemented where in each row you have one user and the user would have multiple columns for different tags. Those tags being their location, time of day, their roles and the level of risk they would have. For example, a user has a librarian role, it is at-site, and they are on the clock. They would be at a low risk level and they would be given full access to do whatever tasks they need. However, say we have the same situation and user, but they are off the clock and they are remote. Therefore, the librarian would not be allowed to access sensitive data because they are at a higher risk of leaking information. Also , since they are working remote there is no guarantee their network is secured. This flexible approach allows for dynamic adaptation of permissions, enhancing security and enabling precise access control based on context.